

KodetoCareer — Technical Requirements Document (TRD)

Version 1.0 | For Engineering Team

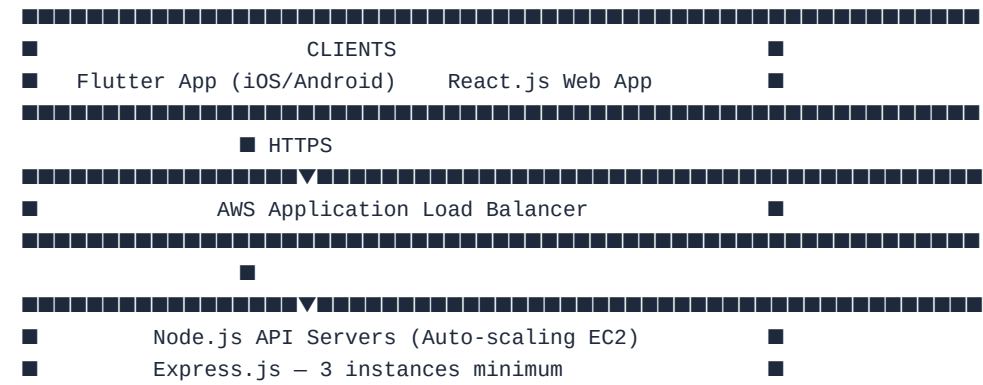
1. System Architecture Overview

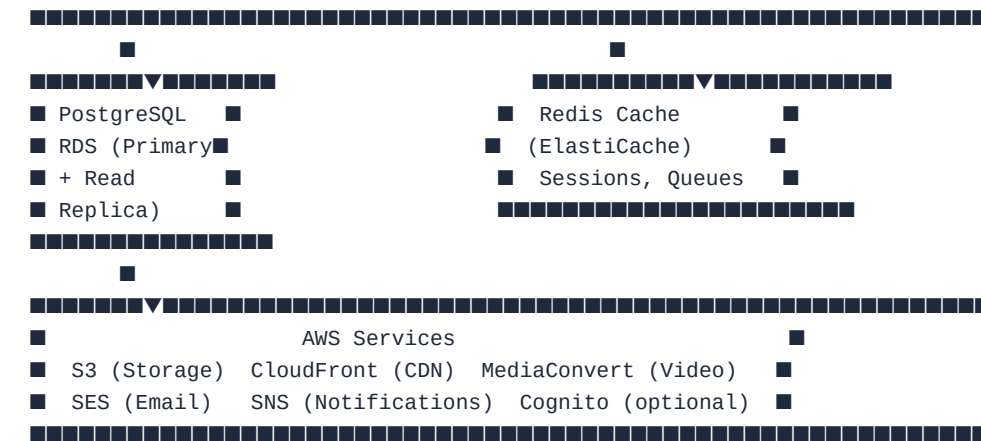
The KodetoCareer platform follows a **multi-tenant, microservice-inspired monolith** architecture for the MVP phase, transitioning to true microservices at scale. This approach balances development speed with long-term scalability.

Architecture Pattern

- **Backend:** Modular Monolith (Node.js / Express.js) with clear domain boundaries
- **Frontend Web:** React.js (Single Page Application)
- **Mobile:** Flutter (iOS + Android, single codebase)
- **Database:** PostgreSQL (primary), Redis (caching & sessions)
- **Storage:** AWS S3 + CloudFront CDN
- **Video Processing:** AWS MediaConvert
- **Authentication:** JWT (access tokens) + Refresh tokens
- **Real-time:** Socket.io for live class chat and notifications
- **Search:** PostgreSQL full-text search (MVP), Elasticsearch at scale

2. Infrastructure Architecture





3. Technology Stack — Detailed

3.1 Backend (API Server)

Component | Technology | Version | Rationale

Runtime | Node.js | 20 LTS | Team familiarity, async I/O performance

Framework | Express.js | 4.x | Minimal, flexible, large ecosystem

ORM | Prisma | 5.x | Type-safe queries, excellent PostgreSQL support

Validation | Zod | 3.x | Runtime schema validation for all API inputs

Authentication | jsonwebtoken | 9.x | Industry standard JWT implementation

File Upload | Multer + AWS SDK | Latest | Efficient multipart upload handling

Email | Nodemailer + AWS SES | Latest | Reliable email delivery

SMS | MSG91 SDK | Latest | Best coverage in India

Queue | Bull (Redis-backed) | Latest | Background jobs: email, video processing, certificates

Logging | Winston + CloudWatch | Latest | Structured logging with remote aggregation

API Docs | Swagger/OpenAPI | 3.0 | Auto-generated from code annotations

Testing | Jest + Supertest | Latest | Unit and integration tests

Process Manager | PM2 | Latest | Production process management

3.2 Frontend Web

Component | Technology | Version | Rationale

Framework | React.js | 18.x | Component ecosystem, team familiarity

State Management | Zustand | 4.x | Lightweight, simpler than Redux for this scale

Routing | React Router | 6.x | Standard SPA routing

UI Components | shadcn/ui | Latest | Accessible, customisable components

Styling | Tailwind CSS | 3.x | Utility-first, rapid development

Data Fetching | React Query (TanStack) | 5.x | Caching, background refetch, optimistic updates

HTTP Client | Axios | 1.x | Interceptors for auth token management

Forms | React Hook Form + Zod | Latest | Performant forms with validation

Charts | Recharts | 2.x | Declarative charts for analytics

Rich Text | TipTap | 2.x | Content editor for notes and announcements

Video Player | Video.js | 8.x | HLS streaming, custom controls

Table | TanStack Table | 8.x | Virtualised tables for large student lists

Build | Vite | 5.x | Fast development builds

3.3 Mobile (Flutter)

Component | Technology | Rationale

Framework | Flutter | 3.x | Single codebase for iOS + Android

State Management | Riverpod | 2.x | Reactive, testable state management

HTTP | Dio | 5.x | Interceptors, retry logic, file upload

Local Storage | flutter_secure_storage | Secure token storage

Push Notifications | firebase_messaging | FCM integration

Video Player | video_player + chewie | Native video playback

PDF Viewer | flutter_pdfview | Native PDF rendering

Offline Support | sqflite | Local SQLite for offline lesson access

Analytics | Firebase Analytics | User behaviour tracking

Crash Reporting | Firebase Crashlytics | Production crash monitoring

Deep Linking | go_router | In-app navigation from notifications

3.4 Database

Component | Technology | Rationale

Primary DB | PostgreSQL 15 | ACID compliance, JSON support, full-text search

Caching | Redis 7 | Session storage, rate limiting, queue

Connection Pool | PgBouncer | Efficient connection management

Migrations | Prisma Migrate | Version-controlled schema changes

Backup | AWS RDS automated | Daily snapshots, point-in-time recovery

4. API Design Specification

4.1 API Standards

- **Style:** RESTful with consistent naming conventions
- **Base URL:** `https://api.kodetocareer.com/v1`
- **Authentication:** Bearer token in Authorization header
- **Request/Response Format:** JSON
- **Versioning:** URL-based (`/v1/`, `/v2/`)
- **Pagination:** Cursor-based for large datasets, offset for admin tables

4.2 Standard Response Envelope

```
// Success
{
  "success": true,
  "data": { ... },
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 347,
    "hasNext": true
  }
}

// Error
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Email is required",
    "details": [
      { "field": "email", "message": "Email is required" }
    ]
  }
}
```

4.3 Core API Endpoints

Authentication

POST	/v1/auth/login	– Login with email/password
POST	/v1/auth/register	– Student self-registration
POST	/v1/auth/verify-email	– Verify email with OTP
POST	/v1/auth/forgot-password	– Send password reset OTP
POST	/v1/auth/reset-password	– Reset password with OTP
POST	/v1/auth/refresh	– Refresh access token
POST	/v1/auth/logout	– Invalidate refresh token

Students

GET	/v1/students	– List students (admin, paginated)
POST	/v1/students/bulk-import	– CSV bulk import
GET	/v1/students/:id	– Get student profile
PUT	/v1/students/:id	– Update student profile
GET	/v1/students/:id/progress	– Get learning progress
GET	/v1/students/:id/analytics	– Get performance analytics
GET	/v1/students/me	– Get own profile (student role)

Colleges

GET	/v1/colleges	– List all colleges
POST	/v1/colleges	– Create college
GET	/v1/colleges/:id	– Get college details
PUT	/v1/colleges/:id	– Update college
GET	/v1/colleges/:id/students	– Students in this college
GET	/v1/colleges/:id/analytics	– College analytics

Courses

GET	/v1/courses	– List courses
POST	/v1/courses	– Create course
GET	/v1/courses/:id	– Get course with modules
PUT	/v1/courses/:id	– Update course
POST	/v1/courses/:id/modules	– Add module
POST	/v1/courses/:id/publish	– Publish course
GET	/v1/courses/:id/enrolled	– Students enrolled in course

Modules & Lessons

POST	/v1/modules/:id/lessons	– Add lesson to module
PUT	/v1/lessons/:id	– Update lesson
DELETE	/v1/lessons/:id	– Delete lesson
POST	/v1/lessons/:id/complete	– Mark lesson complete (student)
POST	/v1/lessons/:id/upload	– Upload video/PDF to lesson

Batches

GET	/v1/batches	– List batches
POST	/v1/batches	– Create batch

GET	/v1/batches/:id	– Get batch details
POST	/v1/batches/:id/enroll	– Enroll students
POST	/v1/batches/:id/import	– Bulk import students to batch
GET	/v1/batches/:id/attendance	– Batch attendance summary

Attendance

POST	/v1/sessions	– Create class session
GET	/v1/sessions/:id	– Get session details
POST	/v1/sessions/:id/attendance	– Mark attendance for session
PUT	/v1/sessions/:id/attendance	– Edit attendance record
GET	/v1/sessions/:batchId	– Get all sessions for batch

Assessments

GET	/v1/quizzes	– List quizzes
POST	/v1/quizzes	– Create quiz
GET	/v1/quizzes/:id	– Get quiz (with questions)
POST	/v1/quizzes/:id/start	– Student starts quiz attempt
POST	/v1/quizzes/:id/submit	– Student submits quiz
GET	/v1/quizzes/:id/results	– Get results and analytics
GET	/v1/quizzes/:id/leaderboard	– Get quiz leaderboard
POST	/v1/assignments	– Create assignment
GET	/v1/assignments/:id/submissions	– Get all submissions
POST	/v1/assignments/:id/submit	– Student submits assignment
PUT	/v1/submissions/:id/grade	– Trainer grades submission

Certificates

POST	/v1/certificates/generate	– Generate certificate(s)
GET	/v1/certificates/:id	– Get certificate
GET	/v1/certificates/verify/:code	– Public verification (no auth)
GET	/v1/students/:id/certificates	– Student's certificates

Placement

GET	/v1/placements	– List placement records
POST	/v1/placements	– Create placement record
PUT	/v1/placements/:id	– Update placement record
GET	/v1/placement/analytics	– Placement analytics
GET	/v1/jobs	– List job opportunities
POST	/v1/jobs	– Post job opportunity
POST	/v1/jobs/:id/interest	– Student marks interest

Notifications

POST	/v1/notifications/announce	– Send announcement
GET	/v1/notifications/me	– Get my notifications
PUT	/v1/notifications/:id/read	– Mark notification read
POST	/v1/notifications/broadcast	– Broadcast to all/batch/college

5. Authentication & Security

5.1 Token Strategy

Access Token:

- JWT signed with RS256 (asymmetric)
- Payload: { userId, role, collegeId, batchIds[], iat, exp }
- Expiry: 15 minutes
- Stored in: memory (web), flutter_secure_storage (mobile)

Refresh Token:

- Opaque random string (64 bytes)
- Stored in: httpOnly cookie (web), secure storage (mobile)
- Expiry: 30 days
- Rotated on each use (refresh token rotation)
- Stored in Redis with user ID mapping for instant revocation

5.2 Security Requirements

TR-SEC-1 — All API endpoints must use HTTPS. HTTP requests must redirect to HTTPS.

TR-SEC-2 — Rate limiting: 100 requests/minute per IP for public endpoints, 1000/minute for authenticated users.

TR-SEC-3 — File uploads must be virus-scanned using ClamAV or AWS GuardDuty before being served.

TR-SEC-4 — All S3 buckets must be private. Content served only through signed CloudFront URLs with 1-hour expiry.

TR-SEC-5 — SQL injection prevention: Prisma ORM with parameterised queries only. No raw SQL string concatenation.

TR-SEC-6 — Sensitive fields in the database (not passwords) that should never appear in API responses: internal notes, payment details.

TR-SEC-7 — Passwords must be hashed with bcrypt (minimum 12 rounds).

TR-SEC-8 — API must implement CORS with a strict whitelist of allowed origins.

TR-SEC-9 — Admin panel must enforce IP allowlisting in production (configurable).

TR-SEC-10 — All admin actions (create, update, delete) must be logged in an audit_logs table.

6. Storage Architecture

6.1 S3 Bucket Structure

kodetocareer-production/

```

■■■ videos/
■   ■■■ {courseId}/{lessonId}/{filename}.mp4
■■■ videos-processed/
■   ■■■ {lessonId}/
■       ■■■ 480p/index.m3u8
■       ■■■ 720p/index.m3u8
■       ■■■ 1080p/index.m3u8
■■■ notes/
■   ■■■ {courseId}/{lessonId}/{filename}.pdf
■■■ certificates/
■   ■■■ {year}/{studentId}/{certificateId}.pdf
■■■ documents/
■   ■■■ {studentId}/offers/{filename}.pdf
■■■ avatars/
■   ■■■ {userId}/{filename}.jpg
■■■ thumbnails/
    ■■■ {courseId}/{filename}.jpg

```

6.2 Video Processing Pipeline

1. Trainer uploads MP4 → Multipart upload to S3 raw bucket
2. S3 event triggers Lambda function
3. Lambda submits MediaConvert job:
 - Output: HLS (480p, 720p, 1080p) + MP4 (720p for download-optional)
 - Thumbnail extraction at 10-second mark
4. MediaConvert completes → processed files written to S3 processed bucket
5. Lambda updates database: lesson.videoStatus = 'READY', lesson.videoUrl = CDN URL
6. Push notification sent to admin/trainer confirming video is live

7. Performance Requirements

7.1 API Performance Targets

Endpoint Category | P50 | P95 | P99

Auth (login/register) | 200ms | 500ms | 1s

Read (get course, lesson) | 100ms | 300ms | 500ms

List (students, batches) | 150ms | 400ms | 800ms

Write (attendance, submission) | 200ms | 500ms | 1s

File upload initiation | 300ms | 600ms | 1s

Report generation (async) | Returns job ID immediately

7.2 Caching Strategy

Data | Cache Location | TTL | Invalidation
Course content | Redis | 1 hour | On course update
Student profile | Redis | 15 minutes | On profile update
Quiz leaderboard | Redis | 5 minutes | On new submission
College analytics | Redis | 30 minutes | On data change
Job board | Redis | 15 minutes | On new posting
Static assets | CloudFront | 1 year | Versioned filenames

8. Background Job Queue

All long-running operations are handled asynchronously via Bull queues:

Queue | Jobs | Worker Count | Priority
video-processing | MediaConvert integration | 2 | Normal
certificate-generation | PDF generation, S3 upload | 3 | High
email-notifications | Transactional + bulk emails | 5 | Normal
sms-notifications | OTP, alerts | 3 | High
report-generation | Analytics PDF/Excel export | 2 | Low
attendance-reminders | Scheduled check, flagging | 1 | Low

9. Real-Time Requirements

Socket.io shall be used for:

- Live class session chat (students ask questions during recorded/live classes)
- Real-time attendance count updates (trainer sees count update as they mark)
- Notification delivery (push to open browser tabs)
- Live quiz countdown (if exam mode is enabled)

Socket.io namespace strategy:

```
/notifications – general user notifications  
/class/{sessionId} – class-specific chat  
/quiz/{quizId}/{attemptId} – quiz session state
```

10. Email Template System

All emails use HTML templates stored in the codebase. Key templates:

Template | Trigger

welcome | Student account created

email-verification | OTP for verification

password-reset | OTP for password reset

enrollment-confirmation | Student enrolled in batch

attendance-low-alert | Attendance drops below threshold

quiz-result | Quiz submitted and scored

assignment-graded | Trainer grades assignment

certificate-ready | Certificate generated

new-job-posting | Relevant job posted

weekly-college-report | Automated Sunday report for college admin

11. Environment Configuration

Environment Variables (Backend)

```
# App
NODE_ENV=production
PORT=3000
API_VERSION=v1
FRONTEND_URL=https://app.kodetocareer.com

# Database
DATABASE_URL=postgresql://user:pass@host:5432/kodetocareer
REDIS_URL=redis://host:6379

# Auth
JWT_PRIVATE_KEY=<RS256 private key>
JWT_PUBLIC_KEY=<RS256 public key>
REFRESH_TOKEN_SECRET=<64 char random>

# AWS
AWS_REGION=ap-south-1
```

```
AWS_ACCESS_KEY_ID=<key>
AWS_SECRET_ACCESS_KEY=<secret>
S3_BUCKET_RAW=kodetocareer-raw-uploads
S3_BUCKET_PROCESSED=kodetocareer-processed
CLOUDFRONT_DOMAIN=cdn.kodetocareer.com
MEDIACONVERT_ENDPOINT=<endpoint>
MEDIACONVERT_ROLE_ARN=<arn>

# Email
SES_REGION=ap-south-1
FROM_EMAIL=noreply@kodetocareer.com

# SMS
MSG91_API_KEY=<key>
MSG91_SENDER_ID=KTOCR

# Push Notifications
FCM_SERVER_KEY=<key>

# Monitoring
SENTRY_DSN=<dsn>
```

12. Deployment Architecture

Production Deployment

AWS Infrastructure (Mumbai Region - ap-south-1):

VPC (10.0.0.0/16)

- ■ ■ ■ Public Subnets (2 AZs)
- ■ ■ ■ Application Load Balancer
- ■ ■ ■ NAT Gateway
- ■ ■ ■ Private Subnets (2 AZs)
- ■ ■ ■ EC2 Auto-scaling Group
- ■ ■ ■ Min: 2 instances, Max: 10
- ■ ■ ■ Instance type: t3.medium (MVP)
- ■ ■ ■ RDS PostgreSQL (Multi-AZ)
- ■ ■ ■ ElastiCache Redis (1 node MVP)
- ■ ■ ■ S3 + CloudFront (global CDN)

CI/CD: GitHub Actions

- ■ ■ ■ Push to main → Run tests → Build Docker image → Deploy to EC2

Containerisation

- Backend: Dockerfile (Node.js 20 Alpine)
- Docker Compose for local development (API + PostgreSQL + Redis)
- ECS Fargate considered for scale (Phase 2)

13. Monitoring & Observability

Tool | Purpose

AWS CloudWatch | Infrastructure metrics, logs

Sentry | Application error tracking (frontend + backend)

Firebase Analytics | Mobile user behaviour

Custom dashboards | Business metrics (active users, quiz submissions/hour)

UptimeRobot | External uptime monitoring + alerts

PgHero | PostgreSQL query performance

14. Disaster Recovery

Scenario | Recovery Approach | RTO | RPO

Single EC2 instance failure | Auto-scaling group replaces | < 5 min | 0

Database corruption | RDS point-in-time restore | < 1 hour | < 5 min

S3 bucket data loss | Cross-region replication to us-east-1 | < 2 hours | < 1 hour

Full region outage | Manual failover to secondary region | < 4 hours | < 1 hour

End of Technical Requirements Document